

Programação Orientada a Objetos

Classes Abstratas e Interfaces: o Primeiro DAO

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Problema: Persistência Repetida

Cliente, Técnico e, em breve, Ordem de Serviço vão precisar das mesmas quatro operações: salvar, buscar por id, listar tudo e remover. Sem um contrato comum, cada tela reinventaria essas quatro operações à sua maneira, com nomes diferentes e bugs diferentes. O problema não é escrever esse código uma vez — é escrever quatro vezes algo que deveria ser um só.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Interface: Contrato Sem Estado

Uma interface declara métodos sem implementá-los e sem guardar nenhum atributo — é contrato puro. Difere da classe abstrata vista na Aula 04: Pessoa guardava nome e telefone e implementava parte do comportamento; uma interface não guarda nada e não implementa nada, apenas promete uma assinatura que qualquer classe implementadora é obrigada a cumprir.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Classe Abstrata × Interface

abstract class Pessoa

guarda estado

nome, telefone

implementa parte

setNome(), toString()

só uma superclasse por vez

interface Dao<T>

nenhum estado

(nenhum atributo)

nenhuma implementação

salvar(), buscarPorId() ...

uma classe implementa várias

Pessoa decide **o que Cliente é**. Dao decide apenas **o que uma classe promete fazer**.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

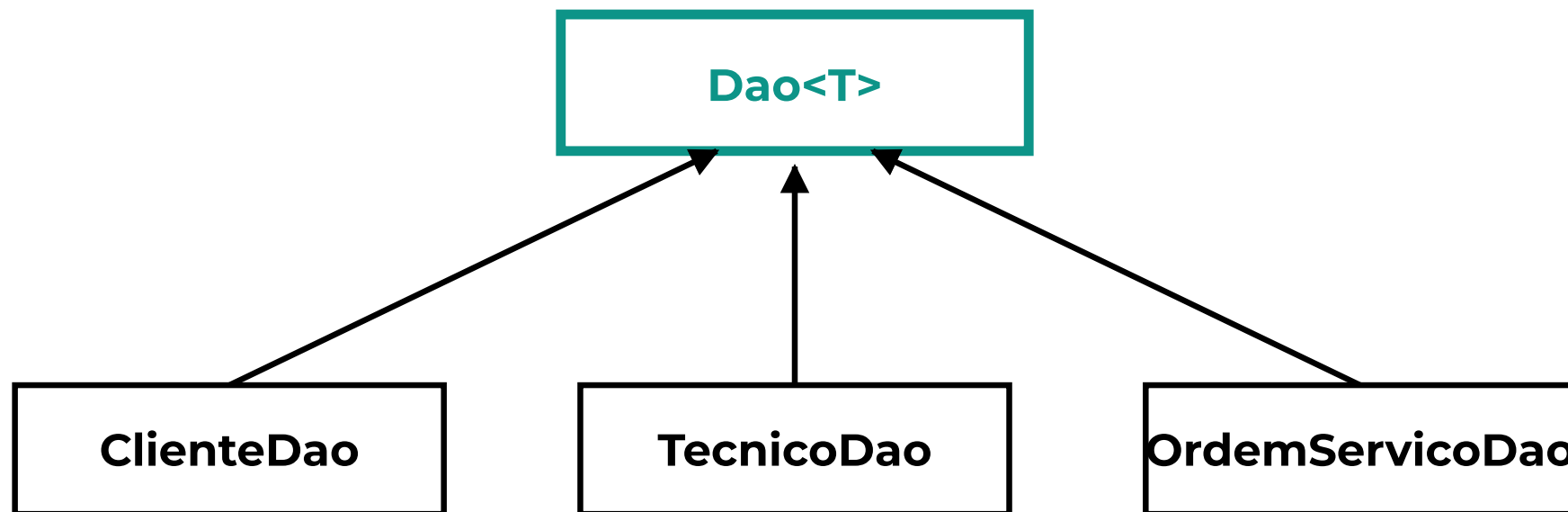
 <https://www.linkedin.com/in/everaldoribeiroads>

Genéricos: Um Contrato, Vários Tipos

Escrever uma interface para cada entidade — ClienteDao, TecnicoDao, cada uma com sua própria assinatura — duplicaria exatamente o problema que a interface deveria resolver. O parâmetro de tipo T permite declarar o contrato uma única vez: Dao<Cliente> e Dao<OrdemServico> são a mesma interface, aplicada a tipos diferentes, sem que uma linha do contrato precise ser reescrita.



Um Contrato, Três Implementações



Cada seta é **implements**. Quem depende de **Dao<T>** não sabe, e não precisa saber, qual delas está do outro lado.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

DaoBase: o Comum Fica Escrito Uma Vez

Salvar, buscar por id, listar e remover seguem exatamente a mesma lógica em qualquer entidade — a única diferença é como ler e definir o identificador de cada tipo. DaoBase implementa `Dao<T>` e escreve essa lógica comum uma única vez, deixando como método abstrato apenas o ponto genuinamente específico: como acessar o id de um T concreto.



Em Memória Hoje, MySQL na Aula 06

DaoBase guarda os registros numa lista, não num banco de dados — é a mesma ideia de TAD já estudada em Estrutura de Dados: o contrato é público, a representação interna é detalhe que pode ser trocado. Na Aula 06 uma nova implementação de `Dao<T>` substitui essa lista por uma tabela do MySQL, e nenhuma linha fora do pacote `dao` vai precisar mudar por causa disso.



A Interface Dao<T>

// Contrato puro: sem atributo, sem corpo de metodo.

```
public interface Dao<T> {
```

```
    T salvar(T objeto);
```

```
    T buscarPorId(int id);
```

```
    List<T> listarTodos();
```

```
    void remover(int id);
```

```
}
```



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

DaoBase: o CRUD Escrito Uma Vez

```
public abstract class DaoBase<T> implements Dao<T> {  
  
    private final List<T> registros = new ArrayList<>();  
    private int proximoId = 1;  
  
    @Override  
    public T salvar(T objeto) {  
        definirId(objeto, proximoId);  
        registros.add(objeto);  
        proximoId++;  
        return objeto;  
    }  
  
    // buscarPorId, listarTodos, remover: mesma ideia, uma vez so.  
    protected abstract void definirId(T objeto, int id);  
    protected abstract int idDe(T objeto);  
}
```



ClienteDao: Só o que É Específico

```
// Não repete salvar, buscar, listar ou remover: tudo isso já
// existe em DaoBase. So ensina onde fica o id de um Cliente.
public class ClienteDao extends DaoBase<Cliente> {

    @Override
    protected void definirId(Cliente objeto, int id) {
        objeto.setId(id);
    }

    @Override
    protected int idDe(Cliente objeto) {
        return objeto.getId();
    }
}
```



O Main Só Conhece o Contrato

```
// A variável é declarada como Dao<Cliente>, não como ClienteDao.  
Dao<Cliente> clienteDao = new ClienteDao();  
  
clienteDao.salvar(new Cliente("Marina Alvarenga", "(32) 98811-4402"));  
clienteDao.salvar(new Cliente("Bruno Kalil", "(32) 99123-7788"));  
  
for (Cliente cliente : clienteDao.listarTodos()) {  
    System.out.println("#" + cliente.getId() + " " + cliente);  
}  
  
System.out.println(clienteDao.buscarPorId(2));
```



Atividades

Exercício 1

Por que `Dao<T>` não pode guardar nenhum atributo, ao contrário de `Pessoa` na Aula 04?

Exercício 2

Se `TecnicoDao` existisse hoje, quais dois métodos ele precisaria implementar, e por quê apenas esses dois?

Exercício 3

O que muda no main quando a Aula 06 trocar a lista de `DaoBase` por uma tabela MySQL?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Criar TecnicoDao, seguindo exatamente o padrão de ClienteDao.
2. Adicionar a Dao<T> um método contar() e implementá-lo em DaoBase.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.

